Turni 568

Birreria 568 Garbatella · Pianificazione turni

Documentazione completa del sistema: analisi architetturale, linee guida per l'utilizzo lato utente e lato amministratore.

Versione 1.0 · Aprile 2026

Indice

PARTE I — ANALISI TECNICA

- 1. Panoramica architetturale
- 2. Struttura del progetto
- 3. Dipendenze esterne
- 4. Schema del database
- 5. Flusso dati e comunicazione
- 6. Sistema notifiche push
- 7. Analisi della sicurezza

PARTE II — GUIDA UTENTE (DIPENDENTE)

- 8. Primo accesso
- 9. Visualizzare il calendario
- 10. Richiedere un cambio turno
- 11. Rispondere a una richiesta di cambio
- 12. Attivare le notifiche push
- 13. Installare l'app (PWA)

PARTE III — GUIDA AMMINISTRATORE

- 14. Gestione dipendenti
- 15. Creazione e modifica turni
- 16. Copiare una settimana
- 17. Gestione richieste di cambio
- 18. Procedure operative consigliate

PARTE IV — OPERAZIONI TECNICHE

- 19. Deploy e variabili d'ambiente
- 20. Manutenzione ordinaria

PARTE I · ANALISI TECNICA

1. Panoramica architetturale

Turni 568 è una Single Page Application (SPA) mobile-first progettata per la gestione settimanale dei turni e delle richieste di cambio del personale di Birreria 568 Garbatella.

Stack tecnologico

Frontend — React 19 + Vite 8

SPA con React Router 7, Tailwind CSS 4, shadcn/ui components. Build ottimizzato da Vite con ESM nativo.

Backend — Supabase (BaaS)

PostgreSQL hosted, Auth email/password, Row Level Security, Realtime WebSocket, Edge Functions (Deno).

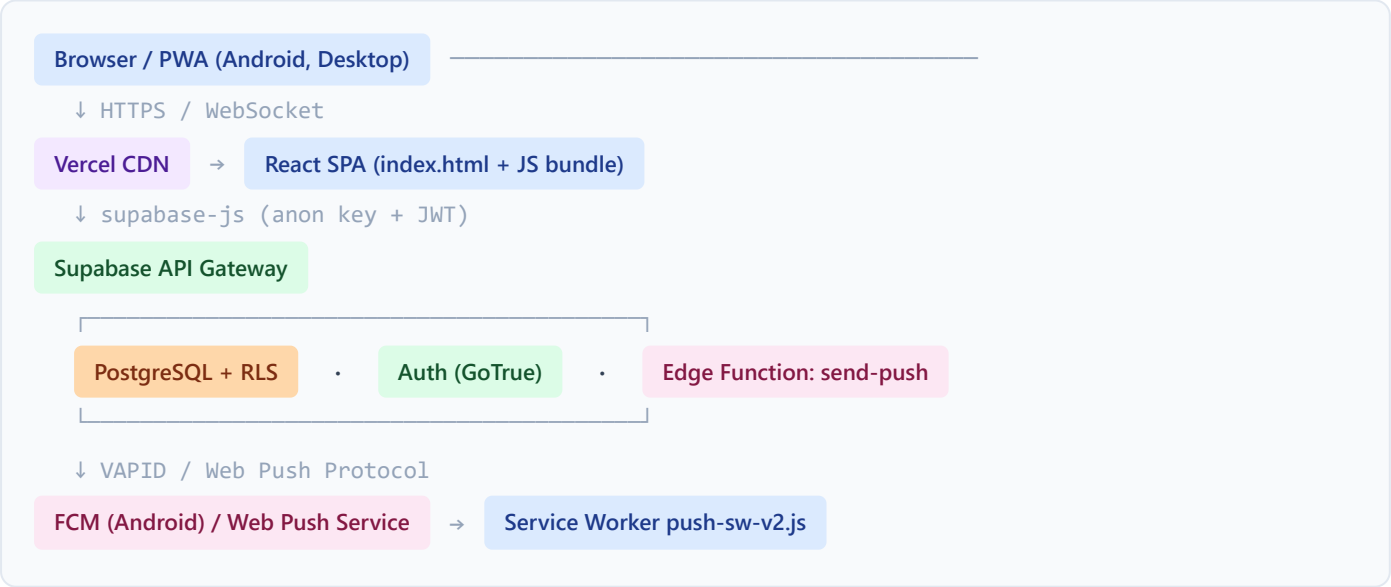
Hosting — Vercel

CDN globale con HTTPS automatico, SPA rewrites, cache headers configurabili via vercel.json.

Push — Web Push API

Service Worker v3 (push-sw-v2.js), VAPID keys, protocollo RFC 8292. Notifiche su Android Chrome e desktop.

Architettura a livelli



PARTE I

2. Struttura del progetto

La codebase è organizzata secondo la convenzione features-first con separazione netta tra logica (hooks/context) e presentazione (pages/components).

Albero dei file principali

```
568-platform/ ├── src/ │ ├── App.jsx ← Router, ProtectedRoute, AdminRoute │ ├── context/ │ │ ├── AuthContext.jsx ← Sessione utente, login/logout │ │ └── RequestsContext.jsx ← Richieste cambio, Realtime, Push │ ├── hooks/ │ │ ├── useCalendar.js ← Turni CRUD, navigazione settimana │ │ ├── useEmployees.js ← Anagrafica dipendenti CRUD │ │ └── usePushNotifications.js ← SW registration, subscribe/unsub │ ├── pages/ │ │ ├── Login.jsx ← Form autenticazione │ │ ├── Calendar.jsx ← Visualizzazione + editing turni │ │ ├── Requests.jsx ← Lista richieste + toggle push │ │ └── Employees.jsx ← Anagrafica (solo admin) │ ├── components/ │ │ ├── Layout.jsx ← Header, bottom nav, logout │ │ ├── calendar/ ← DayRow, EmployeeBadge │ │ ├── modals/ ← ShiftModal, SwapModal, CopyWeekModal │ │ └── employees/ ← EmployeeSheet, DeleteConfirmDialog │ └── lib/ │ ├── supabase.js ← Singleton client + isConfigured guard │ └── utils.js ← cn(), formatters ├── public/ │ ├── push-sw-v2.js ← Service Worker notifiche │ ├── manifest.webmanifest ← PWA manifest │ └── logo.png / logo-badge.png ← Logo Birreria 568 ├── supabase/ │ ├── schema.sql ← Tabelle + RLS + trigger │ ├── push_notifications.sql ← Tabella + RPC subscription │ ├── swap_request_workflow.sql ← RPC create/respond swap │ └── functions/send-push/ │ └── index.ts ← Edge Function Deno (Web Push) └── vercel.json ← SPA rewrites + no-cache headers
```

Pattern architetturali applicati

Pattern	Dove	Motivazione
Context API (Provider pattern)	AuthContext, RequestsContext	Stato globale senza librerie esterne
Custom Hook	useCalendar, useEmployees, usePushNotifications	Separazione logica di business dalla UI
Guard Components	ProtectedRoute, AdminRoute	Protezione route lato client
Security Definer RPC	create_swap_request, respond_to_swap_request, register_push_subscription	Bypass RLS controllato lato server
Graceful Degradation	isSupabaseConfigured, mock data	Sviluppo offline senza backend
Optimistic / Timeout Guard	withTimeout() in AuthContext, RequestsContext	Prevenzione loop infinito su mobile

PARTE I

3. Dipendenze esterne

Catalogo completo di librerie e servizi, con versione, ambito e livello di criticità.

Dipendenze runtime (browser)

Pacchetto	Versione	Ruolo	Criticità
@supabase/supabase-js	2.100.1	Client DB, Auth, Realtime, Functions	🔴 Critico
react + react-dom	19.2.4	Framework UI	🔴 Critico
react-router-dom	7.13.2	Routing SPA	🔴 Critico
@base-ui/react	1.3.0	Dialog, Primitivi UI headless (shadcn)	🟡 Importante
lucide-react	1.7.0	Icone SVG	🟢 Bassa
tailwindcss	4.2.2	CSS utility framework	🟡 Importante
class-variance-authority	0.7.1	Varianti stile componenti	🟢 Bassa
clsx + tailwind-merge	2.1.1 / 3.5.0	Merge classi CSS condizionali	🟢 Bassa
@fontsource-variable/geist	5.2.8	Font Geist (self-hosted)	🟢 Bassa
tw-animate-css	1.4.0	Animazioni CSS aggiuntive	🟢 Bassa

Dipendenze Edge Function (Deno)

Pacchetto	Versione	Ruolo
@supabase/supabase-js (npm:)	2.100.1	Client con service role key per accesso dati
web-push (npm:)	3.6.7	Invio notifiche push via protocollo VAPID/RFC 8292

Servizi cloud esterni

Servizio	Piano	Funzione
Supabase	Free tier	Auth, PostgreSQL, Realtime, Edge Functions, Storage
Vercel	Hobby	Hosting SPA, CDN, HTTPS, domain, deploy CI/CD
FCM / GCM (Google)	Gratuito	Delivery notifiche push su Android Chrome
GitHub	Free	Source control (MatVitale6/568-platform)

⚠️ **Limiti piano gratuito Supabase**

500 MB DB, 5 GB bandwidth, 500.000 auth requests/mese, Edge Functions 500.000 invocazioni/mese. Ampiamente sufficiente per un team piccolo (<20 dipendenti). Monitorare da Dashboard → Reports.

PARTE I

4. Schema del database

Il database PostgreSQL ospitato su Supabase contiene 6 tabelle pubbliche con Row Level Security abilitata su tutte. Le operazioni sensibili passano per funzioni RPC con SECURITY DEFINER.

Entità e relazioni

Tabella	Descrizione	Chiavi import.
profiles	Profilo applicativo dell'utente. Separato da auth.users per maggiore controllo.	id (UUID), auth_user_id → auth.users
employees	Dati anagrafici del dipendente (codice fiscale, telefono, contratto)	profile_id → profiles (1:1)
shifts	Giornata lavorativa. Una riga per data, può essere aperta o chiusa.	id, work_date (unique)
shift_assignments	Assegnazione dipendente a turno (con flag turno parziale)	shift_id → shifts, employee_id → profiles
swap_requests	Richieste di cambio turno tra dipendenti	shift_id, requester_id, target_employee_id → profiles
push_subscriptions	Endpoint push browser per notifiche. Max 1 attiva per profilo.	profile_id → profiles, unique idx (profile_id) WHERE active

Politiche di accesso per ruolo

Tabella	SELECT	INSERT	UPDATE	DELETE
profiles	TUTTI AUTH	ADMIN	SELF / ADMIN	—
employees	TUTTI AUTH	ADMIN	ADMIN	ADMIN
shifts	TUTTI AUTH	ADMIN	ADMIN	ADMIN
shift_assignments	TUTTI AUTH	ADMIN	ADMIN	ADMIN
swap_requests	PROPRIE / ADMIN	VIA RPC	VIA RPC	—
push_subscriptions	PROPRIE / ADMIN	VIA RPC	VIA RPC	PROPRIE / ADMIN

Funzioni RPC (Security Definer)

- ✓ **create_swap_request** — Validazione business logic (requester nel turno, target non già assegnato, nessuna pendente duplicata)
- ✓ **respond_to_swap_request** — Accetta/rifiuta e aggiorna shift_assignments se accettata

- ✅ **register_push_subscription** — Registra endpoint, disattiva subscription precedenti dello stesso profilo
- ✅ **disable_push_subscription** — Disiscrive tutte le subscription attive del profilo corrente

PARTE I

5 & 6. Flusso dati e notifiche push

Come i dati fluiscono dall'utente al backend e come le notifiche raggiungono i dispositivi.

Flusso autenticazione

1. **Inserimento credenziali**
   L'utente inserisce email e password in Login.jsx. La chiamata va a `supabase.auth.signInWithPassword()`.
2. **Emissione JWT**
   Supabase Auth (GoTrue) restituisce un JWT access token (1h) + refresh token. Vengono salvati in localStorage dal client SDK.
3. **Caricamento profilo**
   AuthContext legge il profilo da `public.profiles` tramite `auth_user_id`. Il ruolo (admin/employee) viene caricato qui.
4. **Accesso ai dati**
   Ogni richiesta DB porta il JWT nell'header Authorization. PostgreSQL RLS usa `auth.uid()` per filtrare le righe accessibili.

Flusso cambio turno con notifica push

1. **Dipendente richiede cambio**
   Tap su giorno nel calendario → SwapModal → seleziona collega → RPC `create_swap_request`.
2. **Notifica al target**
   RequestsContext chiama `supabase.functions.invoke('send-push')` passando l'ID del collega target.
3. **Edge Function send-push**
   Valida JWT manualmente, legge endpoint push del target da `push_subscriptions`, invia Web Push tramite VAPID.
4. **Service Worker riceve push**
   `push-sw-v2.js` mostra la notifica di sistema. Click apre l'app su `/requests`.
5. **Aggiornamento in tempo reale**
   Il canale Realtime WebSocket (`swap_requests`) aggiorna la lista in RequestsContext su tutti i client connessi.

Sincronizzazione dati (strategia multi-layer)

Meccanismo	Frequenza	Componente
Realtime WebSocket	Evento (push)	RequestsContext (swap_requests)
Polling silenzioso	ogni 15 s	RequestsContext + useCalendar
Visibility change	Al focus tab	RequestsContext

Meccanismo	Frequenza	Componente
Timeout difensivo	10 s max	AuthContext, RequestsContext (withTimeout)

PARTE I

7. Analisi della sicurezza

Valutazione rispetto alle principali categorie OWASP Top 10. Severità: 🔴 Alta · 🟡 Media · 🟢 Bassa.

Punti di forza

- ✅ RLS abilitata su tutte le 6 tabelle pubbliche — nessuna tabella esposta senza policy
- ✅ Funzioni Security Definer con `set search_path = public` — prevenzione SQL injection tramite search_path manipulation
- ✅ JWT validato manualmente nell'Edge Function (`supabase.auth.getUser(token)`)
- ✅ Anon key in frontend è pubblica per design: RLS garantisce che non si possa leggere/scrivere dati altrui
- ✅ Credenziali sensibili (VAPID private key, service role key) solo in variabili d'ambiente server-side
- ✅ .env.local e cartella supabase/ escluse da git (.gitignore) e da Vercel deploy (.vercelignore)
- ✅ Route admin protetta sia lato client (AdminRoute) sia lato DB (RLS `is_admin()`)
- ✅ Check duplicati in `create_swap_request` — nessuna richiesta doppia pendente
- ✅ Subscription push: una sola attiva per profilo, vincolo indice univoco parziale

Vulnerabilità identificate e raccomandazioni

ID	Problema	Rischio	Raccomandazione
S-01	CORS `Access-Control-Allow-Origin: *` nell'Edge Function send-push	🟡 Media	Restringere all'origin di produzione es. `https://568-platform.vercel.app`
S-02	send-push non verifica che il caller sia autorizzato a notificare i profileIds richiesti	🟡 Media	Validare che profileIds siano colleghi del caller oppure limitare l'invocazione al contesto delle richieste di cambio
S-03	Content-Security-Policy assente in vercel.json	🟡 Media	Aggiungere header CSP con `default-src 'self'`, `connect-src` per supabase URL, `script-src 'self'`
S-04	BASE_URL hardcoded in push-sw-v2.js (`https://568-platform.vercel.app`)	🟢 Bassa (funzionale)	Usare path relativo `/logo.png` se l'app rimane sullo stesso dominio del service worker
S-05	Credenziali mock hardcodate in Login.jsx per modalità demo	🟢 Bassa (mock mode disabilitato in prod)	Rimuovere o spostare in variabile d'ambiente quando la modalità mock viene deprecata
S-06	Il campo `color` dei profili viene inserito direttamente in `style={{backgroundColor}}` senza validazione formato	🟢 Bassa (browser moderni bloccano CSS injection)	Validare che il colore sia un hex `#RRGGBB` valido prima del salvataggio in EmployeeSheet

ID	Problema	Rischio	Raccomandazione
S-07	useEmployees non usa withTimeout() a differenza di useCalendar	🟢 Bassa (UX, non security)	Aggiungere timeout di 10s per consistenza e prevenire hang su mobile

OWASP Top 10 — Mapping

OWASP	Categoria	Stato
A01	Broken Access Control	✅ RLS + RPC Security Definer coprono accesso ai dati
A02	Cryptographic Failures	✅ HTTPS end-to-end, JWT firmati da Supabase, VAPID per push
A03	Injection (SQL/XSS)	⚠️ SQL: OK tramite parametrized queries. XSS: nessun dangerouslySetInnerHTML; CSS injection minore (S-06)
A04	Insecure Design	⚠️ S-02: push send non verifica autorizzazione mittente
A05	Security Misconfiguration	⚠️ CORS wildcard (S-01), CSP assente (S-03)
A06	Vulnerable Components	✅ `npm audit`: 0 vulnerabilità. Dipendenze aggiornate.
A07	Auth Failures	✅ Supabase Auth gestisce sessioni, JWT, refresh automatico
A08	Software Integrity	✅ No CDN esterne, font self-hosted, build deterministica
A09	Logging / Monitoring	⚠️ Logging solo via Supabase Dashboard e Vercel Analytics (nessun alert configurato)
A10	SSRF	✅ Nessuna richiesta proxy server-side verso URL utente-controllati

PARTE II · GUIDA UTENTE

8. Primo accesso

Come accedere all'applicazione per la prima volta come dipendente.

🔗 **URL dell'applicazione**

L'app è accessibile via browser da smartphone o computer. L'URL di produzione verrà comunicato dall'amministratore. Si consiglia di **aggiungere alla home screen** per un accesso rapido (Sezione 13).

Come effettuare il login

1. **Apri il browser e vai all'URL dell'app**
   Usa Chrome (Android consigliato) o Safari (iPhone). Vedrai la schermata con il logo Birreria 568.
2. **Inserisci la tua email e la password**
   Le credenziali ti vengono fornite dall'amministratore tramite email di invito. La password può essere cambiata dal tuo profilo Supabase.
3. **Tocca "Accedi"**
   Se le credenziali sono corrette, verrai reindirizzato al Calendario. La sessione rimane attiva anche chiudendo il browser.

⚠️ **Password dimenticata**

Al momento non esiste una schermata "password dimenticata" integrata nell'app. Contatta l'amministratore per il reset.

9. Visualizzare il calendario

Il calendario è la schermata principale. Mostra i turni settimana per settimana.

Navigazione

Azione	Come farlo
Settimana precedente	Tasto ← in alto a sinistra
Settimana successiva	Tasto → in alto a destra
Mese e numero settimana	Visualizzato al centro (es. "Aprile 2026 · Settimana 15")

Interpretare una riga del calendario

Elemento	Significato
Data e nome giorno	Lunedì–Domenica della settimana visualizzata
Badge colorato con iniziale	Dipendente assegnato a quel turno
Badge con contorno tratteggiato	Turno **parziale** (il dipendente lavora solo parte della giornata)
Riga grigia "Chiuso"	Il locale è chiuso quel giorno
Punto rosso su badge	Esiste una richiesta di cambio turno pendente per quel giorno
Domenica e Lunedì	Giornate di chiusura predefinita (evidenziate in grassetto)

Cosa può fare un dipendente dal calendario

- ✅ Vedere tutti i turni della settimana (propri e dei colleghi)
- ✅ Richiedere un cambio turno toccando il proprio nome in un giorno
- ❌ Non può modificare i turni (solo admin)
- ❌ Non può segnare la chiusura del locale (solo admin)

PARTE II · GUIDA UTENTE

10. Richiedere un cambio turno

Come avanzare una richiesta di cambio a un collega.

1. **Apri il calendario e vai alla settimana desiderata**
   Naviga con le frecce fino a trovare il giorno in cui vuoi scambiare il turno.
2. **Tocca il tuo badge nel giorno**
   Il tuo badge ha il colore e l'iniziale del tuo nome. Toccarlo apre la finestra "Richiedi cambio turno".
3. **Seleziona un collega disponibile**
   La lista mostra solo i colleghi non già in turno quel giorno. Toccane uno per selezionarlo (evidenziato con il suo colore).
4. **Tocca "Conferma"**
   La richiesta viene inviata. Il collega riceverà una notifica push (se attivata) e potrà accettare o rifiutare.

✅ **Dopo l'invio**

Vedrai un messaggio di conferma. La richiesta appare nella sezione "Richieste" con stato `pending`. Puoi inviare solo una richiesta pendente per ogni giorno/collega.

⚠️ **Non trovi il tuo nome nel calendario?**

Significa che non sei assegnato a quel turno. Contatta l'amministratore.

PARTE II · GUIDA UTENTE

11. Rispondere a una richiesta

Come accettare o rifiutare quando un collega ti chiede di scambiare il turno.

🔔 **Ricevi una notifica**

Se le notifiche push sono attive, riceverai una notifica di sistema anche con il browser chiuso. Toccala per aprire direttamente la sezione Richieste.

1. **Vai nella sezione "Richieste" (tab in basso)**
   Il pallino rosso sul tab indica quante richieste pendenti hai.
2. **Trova la richiesta con stato PENDING che ti riguarda**
   Le richieste sono ordinate dalla più recente. Le card mostrano richiedente, collega target, data del turno e stato.
3. **Tocca "Accetta" o "Rifiuta"**
   — **Accetta**: il turno viene automaticamente spostato sulla tua riga nel calendario.
   — **Rifiuta**: la richiesta viene chiusa, nessuna modifica al calendario.

Stati delle richieste

Badge	Significato
PENDING	In attesa di risposta dal collega target
ACCEPTED	Accettata — il turno è stato scambiato
REJECTED	Rifiutata — turno invariato
CANCELLED	Annullata dall'amministratore

PARTE II · GUIDA UTENTE

12. Attivare le notifiche push

Ricevi notifiche in tempo reale anche con l'app chiusa.

1. **Vai in "Richieste" → sezione "Notifiche browser"**
   In cima alla pagina Richieste trovi il pannello notifiche con il pulsante "Attiva notifiche".
2. **Tocca "Attiva notifiche"**
   Il browser mostra un popup di sistema che chiede il permesso per ricevere notifiche. Tocca **Consenti**.
3. **Verifica che lo stato sia "Attivo"**
   Il pulsante diventa "Disattiva notifiche" e lo stato browser mostra "granted".

Risoluzione problemi notifiche

Problema	Soluzione
Il pulsante non appare	Il browser non supporta Web Push (es. Safari iPhone < 16.4 o browser in-app di Instagram/WhatsApp). Usa Chrome.
Permesso negato (blocked)	Vai nelle impostazioni di Chrome → Impostazioni sito → Notifiche → sblocca il dominio dell'app.
Notifiche non arrivano in background	Assicurarsi che le notifiche non siano in modalità silenziosa nelle impostazioni Android.
Le notifiche arrivano in ritardo	Normale se il servizio push è in coda. Su Android controlla che la batteria non stia in "risparmio energetico aggressivo".

⚠️ **Un solo dispositivo alla volta**
Le notifiche sono attive sull'ultimo dispositivo nel quale hai attivato. Se accedi da un nuovo dispositivo e riattivi le notifiche, il vecchio dispositivo non le riceve più.

PARTE II · GUIDA UTENTE

13. Installare l'app (PWA)

Aggiungi Turni 568 alla home screen per usarla come un'app nativa, senza browser.

Android (Chrome)

1. Apri il sito in Chrome
2. Tocca i tre puntini (⋮) in alto a destra
3. Seleziona "Aggiungi a schermata Home"
4. Conferma con "Aggiungi"
   L'icona con il logo Birreria 568 appare sul tuo telefono.

iPhone (Safari)

1. Apri il sito in Safari
2. Tocca l'icona di condivisione (□↑)
3. Scorri e tocca "Aggiungi a Home"
4. Tocca "Aggiungi"
   ⚠️ Le notifiche push su iPhone richiedono iOS 16.4+.

✅ **Vantaggi PWA**

Si apre a schermo intero (senza barra URL), funziona più velocemente dopo il primo caricamento, e riceve le notifiche push come una normale app installata.

14. Gestione dipendenti

La sezione Dipendenti è accessibile solo agli utenti con ruolo **ADMIN**. Permette di creare, modificare, eliminare profili e inviare inviti.

Creare un nuovo dipendente

1. **Vai in "Dipendenti" (tab in basso con icona persone)**
2. **Tocca "+ Crea profilo" in alto a destra**
3. **Compila il form**
   Campi richiesti: **Nome completo**, **Email**, **Codice fiscale**, **Telefono**.
   Opzionale: data fine contratto, colore badge.
4. **Salva**
   Il profilo viene creato. Il dipendente non ha ancora accesso all'app finché non viene inviato l'invito.

Inviare l'invito di accesso

> **Come funziona l'invito**
> L'invito crea l'utente in Supabase Auth con l'email inserita e invia un'email di set-up password. Il dipendente clicca il link nell'email, imposta la password e può accedere all'app.

1. **Trova il dipendente nella lista**
2. **Tocca "Invia invito"**
   Il pulsante appare solo se il dipendente non è ancora stato invitato (`invited: false`). Dopo l'invio, scompare per 3 secondi e il campo "Invitato" diventa ✓.

Modificare o eliminare un dipendente

Azione	Come	Effetto
Modifica	Tocca la riga del dipendente → si apre il pannello laterale con il form precompilato → modifica e salva	Aggiorna profilo e riga employees su DB
Elimina	In fondo al pannello laterale → "Elimina profilo" → conferma nel dialogo	Rimuove profilo, assignments, richieste associate (CASCADE). Irreversibile.

> **Attenzione all'eliminazione**
> L'eliminazione di un dipendente cancella anche i suoi turni assegnati, le sue richieste di cambio e la subscription push. Verifica di aver rimosso prima le assegnazioni in corso.

15. Creazione e modifica turni

Solo l'admin può aggiungere o modificare turni. I dipendenti vedono ma non possono modificare.

Aggiungere o modificare un turno

1. **Vai nel Calendario e naviga alla settimana desiderata**
2. **Tocca una riga giorno (come admin vedi il tasto "+" o la riga cliccabile)**
   Si apre la modale **Modifica turno** con la lista di tutti i dipendenti.
3. **Seleziona i dipendenti in turno**
   Tocca ogni nome per aggiungerlo/rimuoverlo. I selezionati appaiono con il loro colore.
4. **Segna turni parziali (opzionale)**
   Dopo aver selezionato un dipendente, appare il toggle "P" (Parziale). Attivalo se lavora solo parte della giornata.
5. **Salva**
   Le modifiche vengono salvate su Supabase e refreshate in tempo reale per tutti gli utenti connessi.

Segnare una giornata di chiusura

1. **Apri la modale del giorno**
2. **Attiva il toggle "Giornata di chiusura" in cima alla lista**
   Tutti i dipendenti vengono deselezionati automaticamente.
3. **Salva**
   Il giorno appare grigio nel calendario con etichetta "Chiuso".

> 📅 **Chiusure predefinite automatiche**
> Domenica, Lunedì, 24–26 e 31 dicembre, 1° gennaio, 1° maggio sono segnati come chiusi di default (logica `isDefaultClosed()` nel calendario). Non richiedono azione manuale.

16. Copiare una settimana

Risparmia tempo replicando i turni di una settimana su un'altra.

1. **Vai nella settimana che vuoi COPIARE (sorgente)**
2. **Tocca il pulsante "Copia settimana" in alto a destra nel Calendario**
   Si apre la modale con le opzioni di copia.
3. **Seleziona la(e) settimana(e) di DESTINAZIONE**
   Puoi copiare su una o più settimane future.
4. **Conferma**
   I turni (con dipendenti e flag parziale) vengono replicati. Le giornate di chiusura vengono copiate come chiuse.

> ⚠️ **Attenzione — sovrascrittura**
> La copia sovrascrive i turni già esistenti nelle settimane di destinazione. Verifica prima che non ci siano turni già inseriti nelle settimane target.

17. Gestione richieste di cambio

L'admin vede TUTTE le richieste di cambio del team, non solo le proprie.

Panoramica admin

- ✅ Accesso a tutte le richieste di tutti i dipendenti, ordinate dalla più recente
- ✅ Possibilità di accettare o rifiutare qualsiasi richiesta ancora pendente (override)
- ✅ Visibilità su data richiesta e data risposta

Intervento manuale su richiesta

Se un collega target non risponde entro un tempo ragionevole, l'admin può:

1. **Trovare la richiesta nella sezione "Richieste"**
2. **Accettare o rifiutare al posto del target**
   La funzione RPC `respond_to_swap_request` verifica che solo il target o l'admin possa rispondere.

> ⚠️ **Notifiche push verso admin**
> L'admin riceve notifiche push anche per le richieste tra dipendenti (se ha attivato le notifiche sul suo dispositivo).

18. Procedure operative consigliate

Best practice per mantenere il sistema sempre organizzato ed efficiente.

Pianificazione settimanale

- ✅ Inserire i turni della settimana con **almeno 3–4 giorni di anticipo** per dare tempo ai dipendenti di pianificare
- ✅ Usare "Copia settimana" come base se i turni sono simili alla settimana precedente
- ✅ Verificare che le chiusure straordinarie (feste, eventi) siano segnate prima dell'inizio settimana
- ✅ Dopo ogni cambio di assegnazione, i turni si aggiornano in tempo reale per tutti — non è necessario notificare manualmente

Onboarding nuovi dipendenti

1. **Creare il profilo in "Dipendenti"**
   Inserire tutti i dati anagrafici e scegliere un colore unico non già usato.
2. **Inviare l'invito via app**
   Il dipendente riceve l'email con il link per impostare la password.
3. **Comunicare l'URL dell'app e consigliare l'installazione PWA**
4. **Far attivare le notifiche push al dipendente**
   Fondamentale per ricevere avvisi di cambio turno.

Gestione fine rapporto

- ✅ Prima di eliminare il profilo: rimuovere le assegnazioni ai turni futuri
- ✅ Gestire le richieste di cambio pendenti che coinvolgono il dipendente
- ✅ Poi procedere all'eliminazione — il profilo viene rimosso in cascade da DB
- ⚠ Disabilitare anche l'utente in Supabase Dashboard → Authentication → Users per revocare l'accesso immediato

PARTE IV · OPERAZIONI TECNICHE

19. Deploy e variabili d'ambiente

Come rimettere in produzione l'app e le variabili necessarie.

Variabili d'ambiente necessarie

Variabile	Dove	Descrizione
`VITE_SUPABASE_URL`	Vercel + .env.local	URL progetto Supabase (es. https://xxx.supabase.co)
`VITE_SUPABASE_ANON_KEY`	Vercel + .env.local	Chiave pubblica anon/public di Supabase
`VITE_VAPID_PUBLIC_KEY`	Vercel + .env.local	Chiave pubblica VAPID per Web Push
`VAPID_PUBLIC_KEY`	Supabase Edge Function secrets	Stessa chiave pubblica VAPID
`VAPID_PRIVATE_KEY`	Supabase Edge Function secrets	Chiave privata VAPID (non condividere mai)
`VAPID_SUBJECT`	Supabase Edge Function secrets	Email contatto VAPID (es. mailto:admin@...)

Mettere in produzione (Vercel)

```
# Dalla cartella del progetto cd C:\Users\bello\Downloads\568-platform npx vercel --prod
```

Le variabili d'ambiente Vercel (`VITE_*`) devono essere già configurate nel pannello Vercel (Project → Settings → Environment Variables). Vengono incluse nel build al deploy.

Aggiornare la Edge Function send-push

```
npx supabase functions deploy send-push --no-verify-jwt --use-api
```

Il flag `--use-api` è necessario su Windows per evitare errori Docker. Il flag `--no-verify-jwt` è necessario perché la funzione fa la validazione JWT manualmente.

Eseguire le migration SQL

Se le tabelle non sono ancora state create, eseguire questi file nell'ordine tramite Supabase Dashboard → SQL Editor:

1. **`supabase/schema.sql`**
   Crea tutte le tabelle principali + RLS + trigger
2. **`supabase/push_notifications.sql`**

Tabella push_subscriptions + RPC register/disable

3. **`supabase/swap_request_workflow.sql`**

   RPC create_swap_request e respond_to_swap_request

4. **`supabase/bootstrap_admin.sql`**

   Crea il primo admin (modificare UUID e dati prima di eseguire)

PARTE IV · OPERAZIONI TECNICHE

20. Manutenzione ordinaria

Operazioni periodiche consigliate per mantenere il sistema efficiente.

Manutenzione mensile

- ✅ Verificare da Supabase Dashboard → Reports che l'utilizzo rimanga nei limiti del piano free
- ✅ Controllare che non ci siano richieste di cambio **pending** rimaste aperte da più di 7 giorni
- ✅ Verificare che le subscription push inattive vengano pulite (il sistema `active: false` le segna automaticamente su 404/410)

Aggiornamento dipendenze

```
# Controlla vulnerabilità npm audit # Aggiorna patch sicure npm audit fix # Visualizza aggiornamenti disponibili npx npm-check-updates -u
```

> ⚠️ **Aggiornamenti Supabase**
> La libreria `@supabase/supabase-js` è critica. Testa sempre localmente prima di deployare aggiornamenti major. Controllare il changelog di Supabase per breaking changes.

Backup e ripristino

- ✅ Supabase piano free include backup automatici del DB con retention di 1 giorno
- ✅ Per export manuale: Supabase Dashboard → Database → Backups oppure pg_dump via connection string
- ✅ Il codice sorgente è su GitHub (MatVitale6/568-platform, branch main) — aggiornare dopo ogni modifica con `git push`

Contatti tecnici e riferimenti

Risorsa	URL / info
Supabase Dashboard	https://supabase.com/dashboard/project/fnwuanftlhbqksldgafe
Vercel Dashboard	https://vercel.com/matvitale6s-projects/568-platform
GitHub Repository	https://github.com/MatVitale6/568-platform
Documentazione Supabase	https://supabase.com/docs
Documentazione Web Push	https://web.dev/push-notifications-overview/